Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 10

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

Documentación del laboratorio Semana 10

Componentes props estado y eventos en React

Reorganicé la interfaz de la clínica veterinaria de la Semana 9 en componentes funcionales y agregué búsquedas, contadores y registro de pacientes. El frontend conserva datos de práctica compatibles con la API de la Semana 8. Las altas permanecen en el estado de React mientras no se conecten con Django.

Ejercicio 1. Recuperar el proyecto e identificar componentes

Partí del frontend Vite de la Semana 9 y ejecuté la aplicación. Identifiqué como componentes separados la tarjeta de medicamento, la lista de pacientes y la lista de atenciones: cada uno tiene una responsabilidad visible y se puede reutilizar o mantener por separado. Creé src/components y trasladé los arreglos de ejemplo a src/data.js.

Estructura del frontend y candidatos a componente

App.jsx · Extracto literal del archivo fuente

```
2  import './App.css'
3  import MedicamentoLista from './components/MedicamentoLista.jsx'
4  import PacienteLista from './components/PacienteLista.jsx'
5  import PacienteFormulario from './components/PacienteFormulario.jsx'
6  import VeterinarioLista from './components/VeterinarioLista.jsx'
7  import AtencionLista from './components/AtencionLista.jsx'
8  import { atenciones, medicamentos as medicamentosBase, pacientes as pacientesBase, propietarios, veterinarios }
          from './data.js'
```

Imagen 1. Extracto literal de App.jsx con los componentes que utiliza la interfaz.

Ejercicio 2. Crear el primer componente funcional

Creé MedicamentoTarjeta.jsx, un componente funcional cuyo nombre empieza con mayúscula, que devuelve el JSX de un medicamento. MedicamentoLista lo muestra varias veces al recorrer el arreglo. Si el nombre de una función se escribe en minúscula en JSX, React la interpreta como una etiqueta HTML y no como un componente importado.

Componente funcional que se reutiliza en la lista

MedicamentoLista.jsx · Extracto literal del archivo fuente

```
19        <div className="medication-list">
20          {visibles.length ? visibles.map((medicamento) => (
21            <MedicamentoTarjeta key={medicamento.id} id={medicamento.id} nombre={medicamento.nombre}
          descripcion={medicamento.descripcion} precio={medicamento.precio} stock={medicamento.stock} />
22          )) : <div className="empty-state medication-empty"><strong>No hay medicamentos</strong><p>Prueba otra
          búsqueda.</p></div>}
```

Imagen 2. Extracto literal de MedicamentoLista.jsx con la lista de componentes hijos.

Ejercicio 3. Pasar datos con props y armar la lista

MedicamentoTarjeta recibe id, nombre, descripción, precio y stock por props desestructuradas. descripcion y stock tienen valores predeterminados. MedicamentoLista entrega cada objeto y usa el id como key para que React identifique los elementos de la lista.

Props desestructuradas y valor por defecto

MedicamentoTarjeta.jsx · Extracto literal del archivo fuente

```
4
5  export default function MedicamentoTarjeta({ id, nombre, descripcion = 'Producto veterinario', precio, stock = 0
         }) {
6    const [agregados, setAgregados] = useState(0)
```

Imagen 3. Extracto literal de MedicamentoTarjeta.jsx con props y valores por defecto.

Ejercicio 4. Crear un componente contenedor con children

Tarjeta recibe un título y el contenido anidado mediante children. Lo utilizo como envoltura del formulario de registro y del equipo médico. En los Templates de Django cumple un papel parecido al bloque de contenido que se inserta dentro de una plantilla contenedora, aunque React construye esa composición en el navegador.

Contenedor reutilizable con children

Tarjeta.jsx · Extracto literal del archivo fuente

```
1  export default function Tarjeta({ titulo, children, className = '' }) {
2    return (
…
4        {titulo && <header className="component-card__header"><h3>{titulo}</h3></header>}
5        <div className="component-card__body">{children}</div>
6      </section>
```

Imagen 4. Extracto literal de Tarjeta.jsx con children.

Ejercicio 5. Componentes dentro de componentes y relación 1:N

AtencionLista agrupa las atenciones por paciente y presenta cada grupo mediante AtencionTarjeta; cada registro anidado se dibuja con DetalleAtencion. AtencionTarjeta calcula el total usando reduce(). Los nombres de la respuesta API, como fecha_realizacion, se leen en el componente y el objeto se pasa completo por props. Las props son de solo lectura: modificar la entrada del hijo puede fallar y también rompe el flujo unidireccional de datos.

Composición anidada y total de una relación

AtencionTarjeta.jsx · Extracto literal del archivo fuente

```
1  function DetalleAtencion({ atencion }) {
2    return (
…
10
11 export default function AtencionTarjeta({ paciente, atenciones = [] }) {
12   const total = atenciones.reduce((suma, atencion) => suma + Number(atencion.monto), 0)
13   return (
```

Imagen 5. Extracto literal de AtencionTarjeta.jsx con el componente anidado y el total calculado.

```
Prueba ejecutada con React en Node.js:
propsFrozen: true
TypeError: Cannot assign to read only property 'stock' of object '#<Object>'
```

La interfaz no modifica las props; los valores que el usuario cambia se guardan en estado local o en el estado del componente padre.

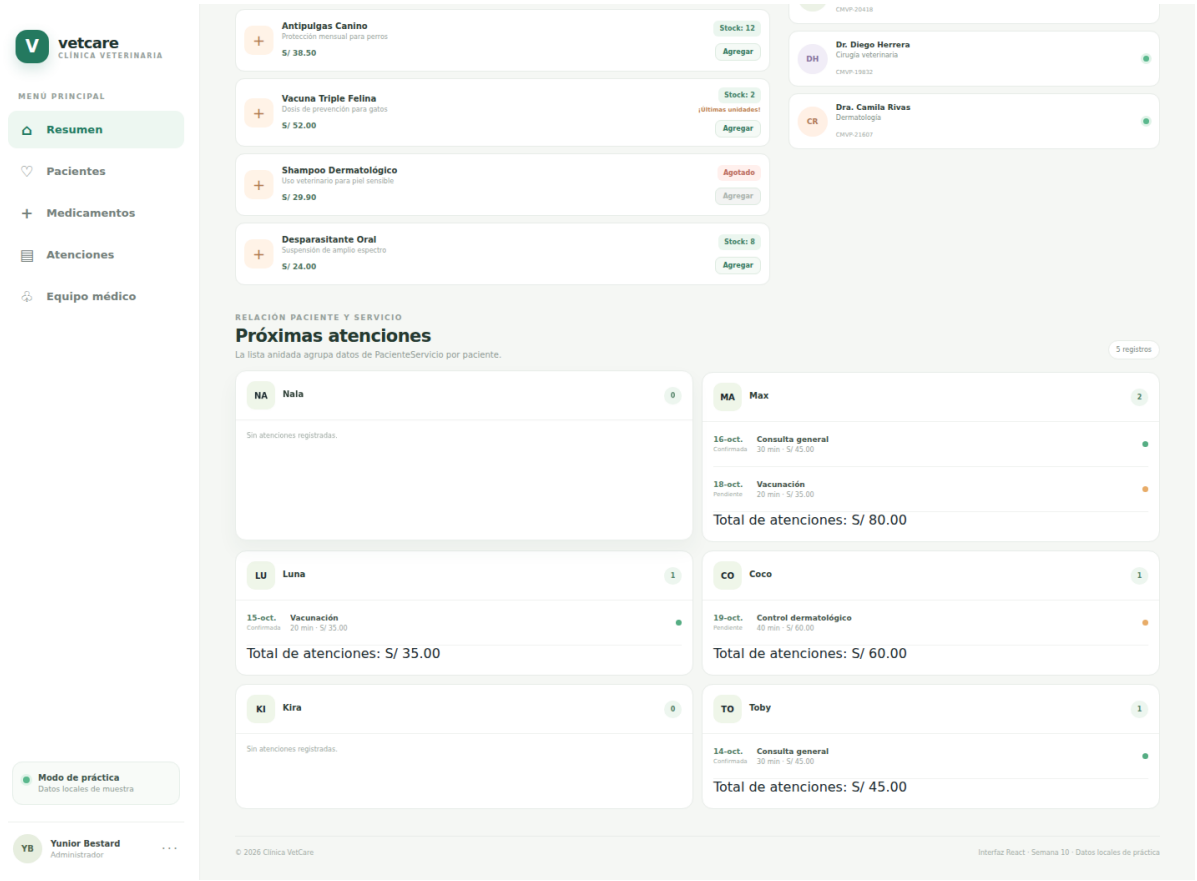


Imagen 6. Captura real del navegador con atenciones agrupadas bajo cada paciente.

Ejercicio 6. Agregar estado y un evento de clic

Cada MedicamentoTarjeta mantiene un contador local con useState. El evento onClick aumenta el contador usando el valor anterior y cada instancia conserva su propio número. La tarjeta muestra el contador cuando es mayor que cero y desactiva el botón de un medicamento agotado. Una variable let no avisa a React y no actualiza la pantalla.

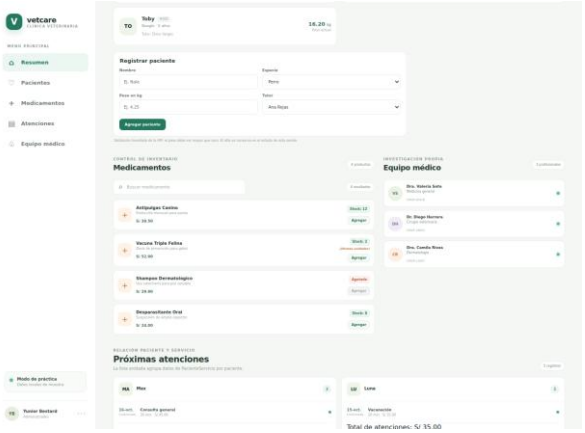


Imagen 7. Antes del clic: botones en cero.

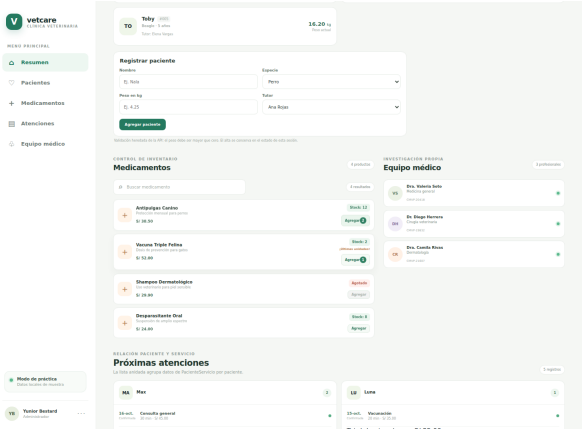


Imagen 8. Después: antipulgas en 2 y vacuna en 1.

Estado local independiente por tarjeta

MedicamentoTarjeta.jsx · Extracto literal del archivo fuente

```
1  import { useState } from 'react'
2
…
5  export default function MedicamentoTarjeta({ id, nombre, descripcion = 'Producto veterinario', precio, stock = 0
          }) {
6    const [agregados, setAgregados] = useState(0)
7    const agotado = Number(stock) <= 0
…
22          {!agotado && stock <= 3 && <span className="low-stock-message">¡Últimas unidades!</span>}
23          <button className="add-counter" type="button" onClick={agregar} disabled={agotado} aria-label={`Agregar
          ${nombre}`}>
24            Agregar{agregados > 0 && <span className="counter-value"> {agregados}</span>}
```

Imagen 9. Extracto literal con useState, el evento de clic y la condición disabled.

Ejercicio 7. Campo controlado que filtra la lista

MedicamentoLista controla el texto de búsqueda con value y onChange. El filtro convierte el texto escrito y el nombre a minúsculas, por lo que no distingue entre mayúsculas y minúsculas. Si no hay coincidencias, aparece un mensaje de estado vacío. También probé el filtro de pacientes con dos nombres diferentes.

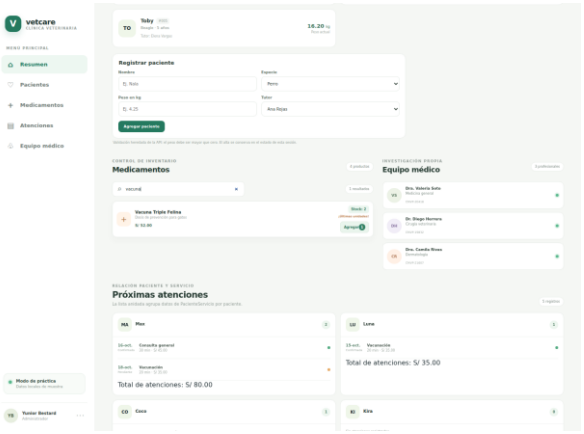


Imagen 10. Búsqueda «vacuna»: un resultado.

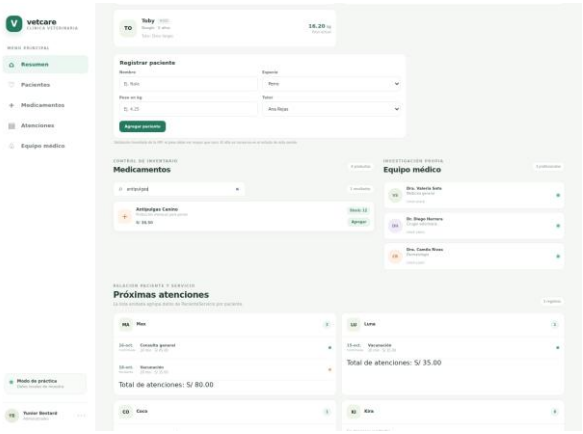


Imagen 11. Búsqueda «antipulgas»: un resultado distinto.

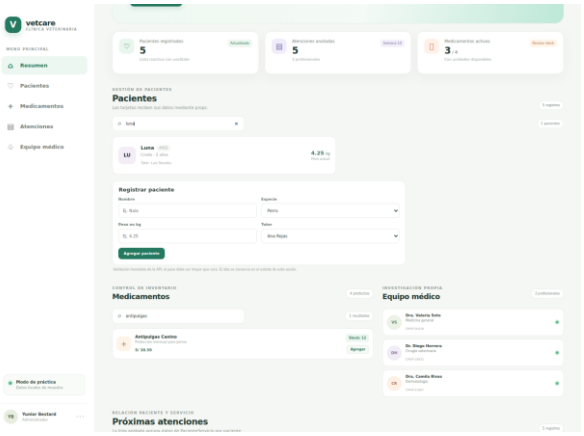


Imagen 12. Búsqueda de paciente «luna».

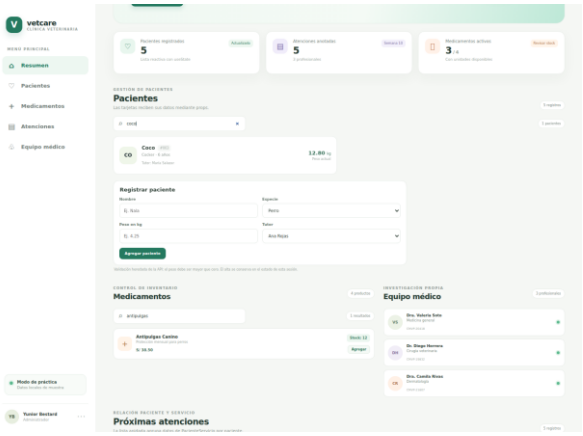


Imagen 13. Búsqueda de paciente «coco».

Búsqueda controlada y filtrado sin distinguir mayúsculas

MedicamentoLista.jsx · Extracto literal del archivo fuente

```
1  import { useMemo, useState } from 'react'
2  import MedicamentoTarjeta from './MedicamentoTarjeta.jsx'
…
5    const [busqueda, setBusqueda] = useState('')
6    const visibles = useMemo(() => medicamentos.filter((medicamento) =>
7      medicamento.nombre.toLocaleLowerCase('es').includes(busqueda.trim().toLocaleLowerCase('es')),
…
14           <span aria-hidden="true">⌕</span>
15           <input type="search" value={busqueda} onChange={(event) => setBusqueda(event.target.value)}
         placeholder="Buscar medicamento" aria-label="Buscar medicamento" />
16         </label>
```

Imagen 14. Extracto literal de la búsqueda controlada en MedicamentoLista.jsx.

Ejercicio 8. Formulario controlado con onSubmit y validación

PacienteFormulario controla nombre, especie, peso y tutor. Al enviar, usa preventDefault y valida el nombre y el peso mayor que cero, conforme a la regla de la API de pacientes de Semana 8. Si el peso vale cero, muestra un mensaje de error. Con datos válidos agrega el registro mediante un arreglo nuevo, limpia los campos y presenta una confirmación. Para guardar en Django aún falta enviar la petición HTTP a la API, tratar su respuesta y actualizar el estado con el registro persistido.

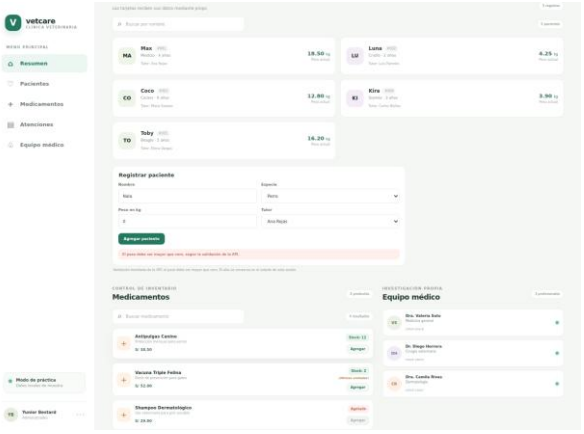


Imagen 15. Envío inválido: el peso cero presenta el mensaje de validación.

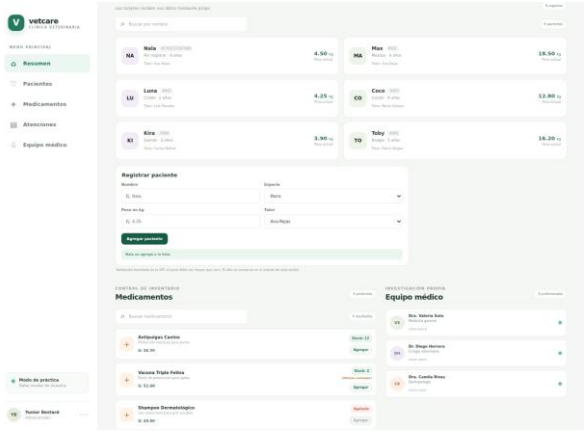


Imagen 16. Envío válido: Nala aparece en la lista y el contador pasa a 6.

Formulario controlado con validación de peso

PacienteFormulario.jsx · Extracto literal del archivo fuente

```
  1  import { useState } from 'react'
  2  import Tarjeta from './Tarjeta.jsx'
…
 17    const enviar = (event) => {
 18      event.preventDefault()
 19      const nombre = campos.nombre.trim()
…
 21      if (!nombre) return setError('Escribe el nombre de la mascota.')
 22      if (!Number.isFinite(peso) || peso <= 0) return setError('El peso debe ser mayor que cero, según la
           validación de la API.')
 23
 24      onAgregar({
 25        id: Date.now(), propietario: Number(campos.propietario), nombre,
…
 34      <Tarjeta titulo="Registrar paciente" className="form-card">
 35        <form className="patient-form" onSubmit={enviar} noValidate>
 36          <label>Nombre
```

Imagen 17. Extracto literal de PacienteFormulario.jsx con el envío, la validación y el alta inmutable.

Ejercicio 9. Identificar los componentes de la investigación propia

Apliqué los elementos del laboratorio a Medicamento, Veterinario, Paciente y PacienteServicio, que conecta al paciente con el servicio atendido. El formulario usa la validación de peso positivo ya implementada por la API de Semana 8.

Concepto	Aplicación en la clínica
Componente de registro	MedicamentoTarjeta muestra nombre, precio y stock.
Botón con estado	Agregar incrementa un contador local por tarjeta.
Relación anidada	AtencionTarjeta presenta PacienteServicio y el Servicio dentro de cada paciente.
Formulario y validación	PacienteFormulario registra nombre, especie y peso; el peso debe ser mayor que cero.

Datos de la investigación y relación intermedia

data.js · Extracto literal del archivo fuente

```
  17
  18  export const medicamentos = [
  19    { id: 1, nombre: 'Antipulgas Canino', descripcion: 'Protección mensual para perros', stock: 12, precio:
          '38.50' },
…
  24
  25  export const veterinarios = [
  26    { id: 1, nombre_completo: 'Dra. Valeria Soto', especialidad: 'Medicina general', colegiatura: 'CMVP-20418' },
…
  37  // Forma de PacienteServicioSerializer: paciente y servicio vienen anidados desde la API.
  38  export const atenciones = [
  39    {
```

Imagen 18. Extracto literal de data.js con las entidades y la relación de atenciones.

Ejercicio 10. Componentes y props de la investigación

La investigación está organizada en archivos funcionales separados: PacienteTarjeta muestra un registro, PacienteLista recibe el arreglo y crea tarjetas, y Tarjeta envuelve contenido con children. MedicamentoLista y VeterinarioLista aplican también map() y key a sus registros.

Componentes de registro, lista y contenedor

PacienteLista.jsx · Extracto literal del archivo fuente

```
   3
   4  export default function PacienteLista({ pacientes = [], propietarios = [] }) {
   5    const [busqueda, setBusqueda] = useState('')
…
  20        <div className="patient-grid">
  21          {visibles.length ? visibles.map((paciente) => (
  22            <PacienteTarjeta key={paciente.id} paciente={paciente}
            propietario={propietariosPorId[paciente.propietario]} />
  23          )) : <div className="empty-state"><strong>No hay registros</strong><p>Prueba con otro nombre.</p></div>}
```

Imagen 19. Extracto literal de PacienteLista.jsx con props, map y key.

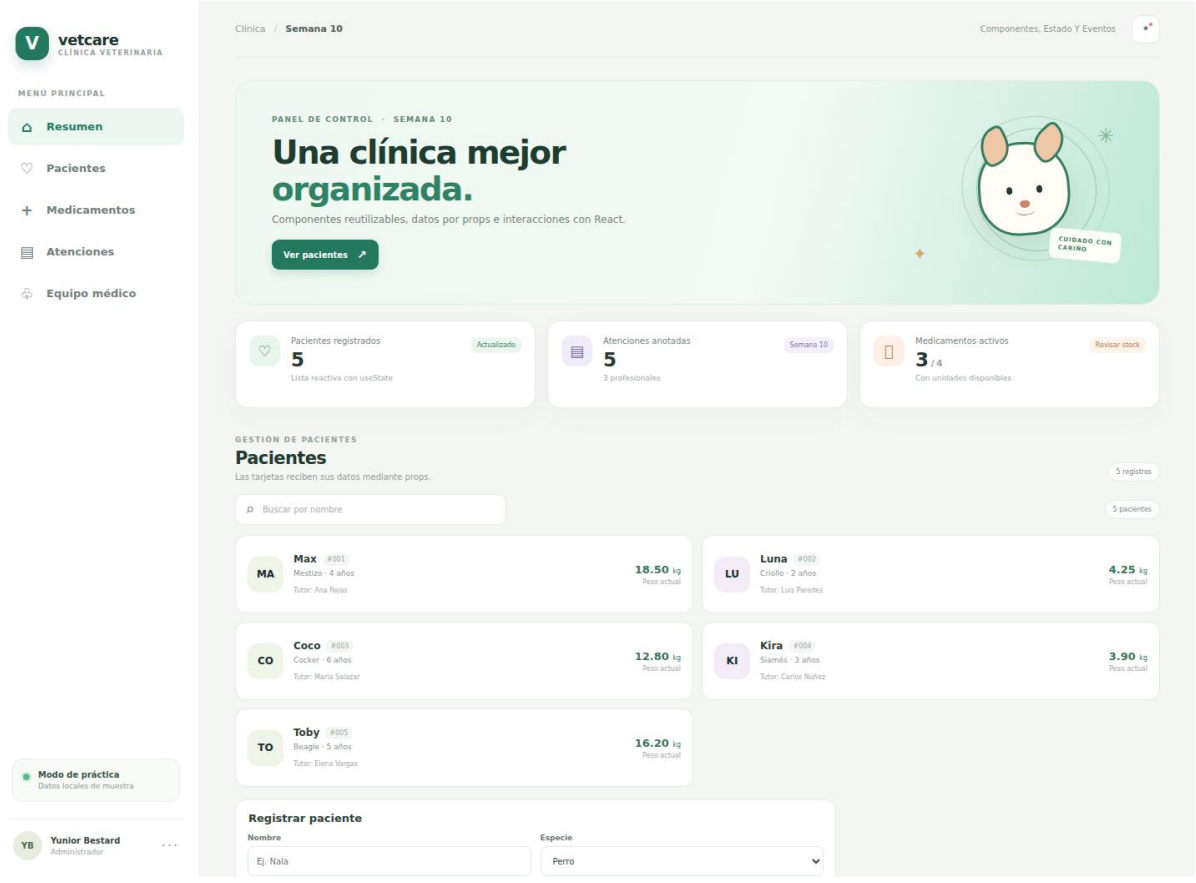


Imagen 20. Captura real del panel con las listas de pacientes y medicamentos.

Ejercicio 11. Estado, eventos y buscador en la investigación

Probé el contador de medicamentos antes y después del clic, además del campo controlado con búsquedas distintas. El contador se mantiene por instancia y las listas se actualizan al cambiar el texto. Ambos controles reflejan el estado actual de React.

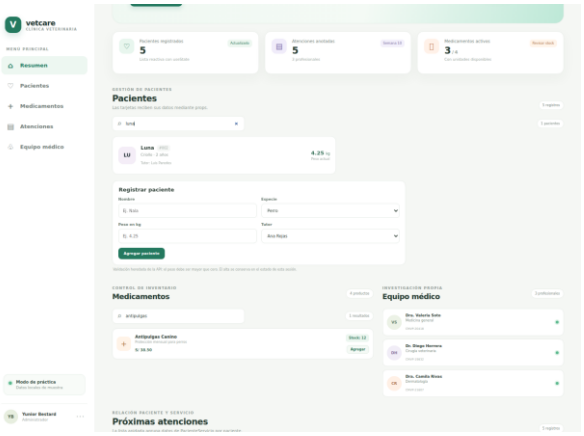


Imagen 21. Resultado de «luna».

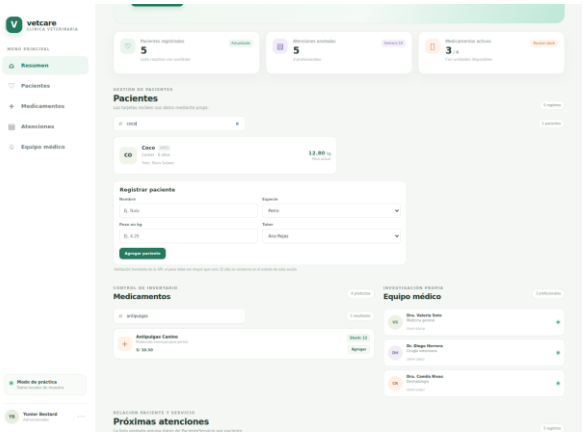


Imagen 22. Resultado de «coco».

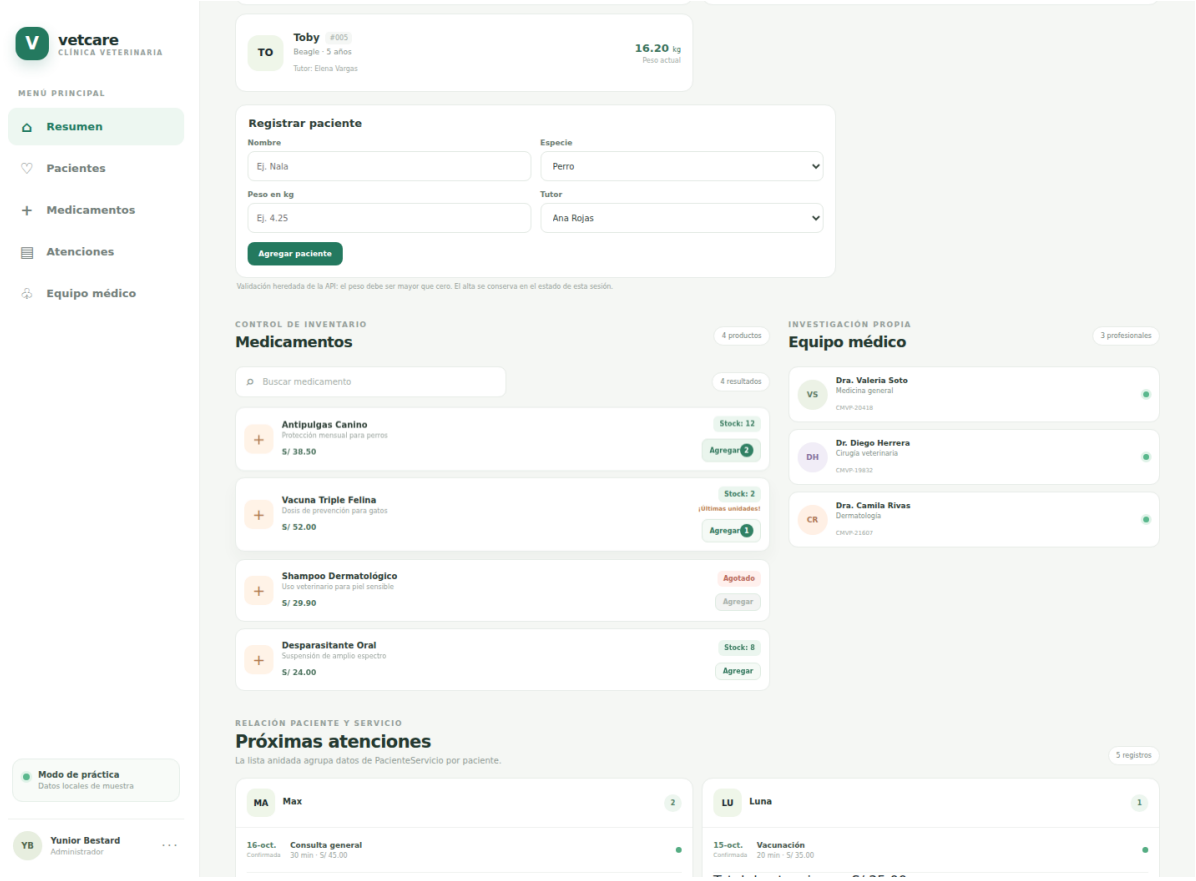


Imagen 23. Captura real con los contadores independientes actualizados.

Eventos de escritura y actualización de estado

PacienteLista.jsx · Extracto literal del archivo fuente

```
  1  import { useMemo, useState } from 'react'
  2  import PacienteTarjeta from './PacienteTarjeta.jsx'
…
  4  export default function PacienteLista({ pacientes = [], propietarios = [] }) {
  5    const [busqueda, setBusqueda] = useState('')
  6    const propietariosPorId = useMemo(() => Object.fromEntries(propietarios.map((propietario) => [propietario.id,
         propietario])), [propietarios])
…
 15            <span aria-hidden="true">⌕</span>
 16            <input type="search" value={busqueda} onChange={(event) => setBusqueda(event.target.value)}
         placeholder="Buscar por nombre" aria-label="Buscar paciente por nombre" />
 17          </label>
```

Imagen 24. Extracto literal del estado y del evento onChange de PacienteLista.jsx.

Ejercicio 12. Relación anidada y formulario en la investigación

AtencionLista crea una tarjeta por paciente e inserta sus atenciones y servicios. El formulario agrega pacientes al mismo arreglo de estado. Probé el envío con peso cero y con peso válido; el primero conserva un mensaje de error y el segundo incorpora el nuevo paciente y limpia los campos.

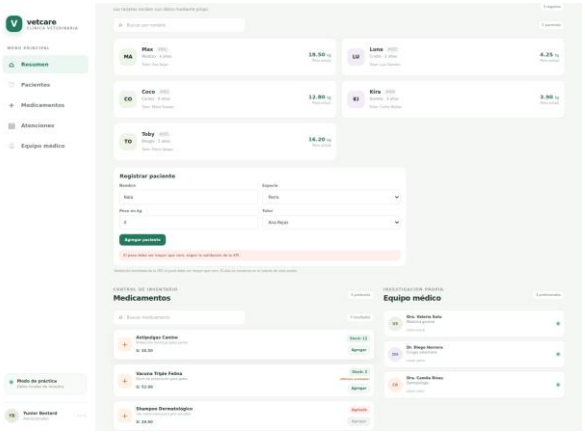


Imagen 25. Validación rechaza peso cero.

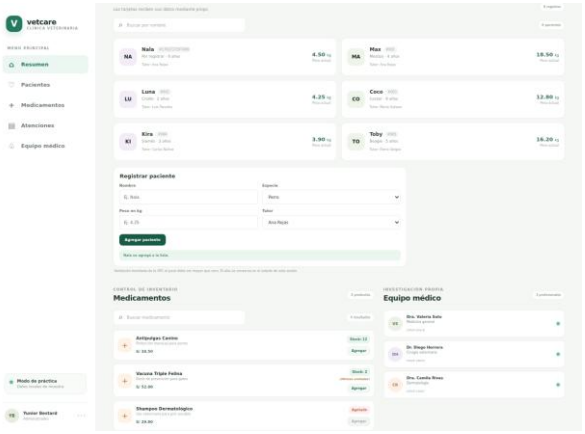


Imagen 26. El paciente nuevo queda visible en la lista.

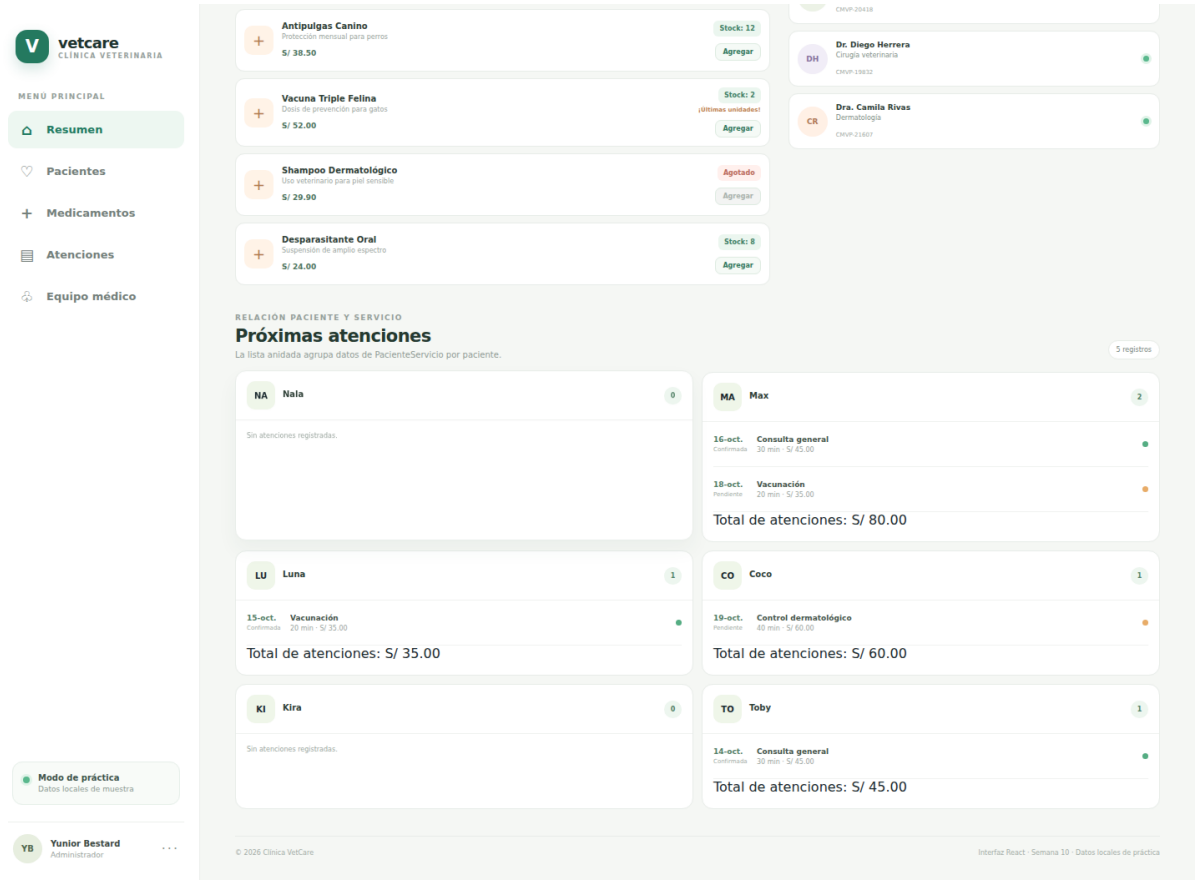


Imagen 27. Captura real de la relación anidada y los totales por paciente.

Atenciones anidadas y formulario de alta

AtencionLista.jsx · Extracto literal del archivo fuente

```
  3  export default function AtencionLista({ pacientes = [], atenciones = [] }) {
  4    const grupos = pacientes.map((paciente) => ({
  5      ...paciente,
…
  8    return <div className="attention-grid">
  9      {grupos.map((paciente) => <AtencionTarjeta key={paciente.id} paciente={paciente}
           atenciones={paciente.atenciones} />)}
 10    </div>
```

Imagen 28. Extracto literal de AtencionLista.jsx con la composición de la relación.

Ejercicio 13. Verificación final de la interfaz

Probé contadores, dos filtros de medicamentos, dos búsquedas de pacientes y los envíos válido e inválido. El navegador no registró advertencias ni errores. ESLint terminó sin errores y Vite compiló 40 módulos. .gitignore excluye node_modules y dist, y esas carpetas no forman parte de la publicación.

```
$ npm run lint
> eslint .
(sin errores; código de salida 0)

$ npm run build
✓ 40 modules transformed
✓ built successfully

$ Consola del navegador
[]
```

npm mostró una advertencia de configuración del entorno sobre http-proxy; no bloqueó los comandos. La verificación del repositorio confirmó que node_modules y dist no están incluidos.

Exclusión de archivos generados

.gitignore · Extracto literal del archivo fuente

```
1  node_modules/
2  dist/
3  .DS_Store
```

Imagen 29. Extracto literal de frontend/.gitignore con node_modules y dist excluidos.

Ejercicio 14. README y publicación en GitHub

Actualicé el README general con la misma estructura de las semanas anteriores y agregué el README de Semana 10. Este último explica cada componente, cómo ejecutar el frontend y qué falta para persistir datos mediante la API. Publiqué Semana_10 en main, conservando las carpetas previas.

README de Semana 10

README.md · Extracto literal del archivo fuente

```
15
16  ## Componentes implementados
17
…
30
31  ## Ejecución
32
…
43  ```bash
44  npm run lint
45  npm run build
```

Imagen 30. Extracto literal del README de Semana 10.

Repositorio del curso: https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales

Commit publicado en main: 4ae930bc4ceb74deb52542afef5ee563a1a47ee3 — Desarrollar laboratorio 10 con componentes y eventos React.

Conclusiones

El frontend quedó organizado en componentes que reciben datos mediante props, componen contenido con children y gestionan cambios con estado y eventos. Las búsquedas y el formulario respondieron a las interacciones probadas; la validación de peso sigue la regla de la API. Lint y build finalizaron correctamente. El siguiente paso técnico para persistir las altas será conectar el formulario con el endpoint Django.